

SokoPulse: An Autonomous Competitor-Aware Dynamic Pricing Engine for Kenyan Distributors and Retailers

1. Introduction

Commerce in Kenya relies heavily on large-scale distributors and mid-level retailers. These businesses handle everything from fresh agricultural produce at Wakulima Market to heavy construction materials like Bamburi Cement. Operating in a high-volume, low-margin environment requires extreme precision. A pricing error of even 2% or 3% can easily wipe out a profitable month, while holding too much stock ties up millions of shillings in capital.

Currently, many distributors still track their pricing manually using Excel spreadsheets. The larger tech-enabled networks rely on closed, proprietary systems that independent distributors cannot access. Even those utilizing heavy legacy ERP systems or standard POS desktop software find themselves limited. These existing systems are fundamentally reactive and isolated. They record internal transactions but remain blind to external market forces. SokoPulse addresses this gap by operating as an autonomous, competitor-aware dynamic pricing engine. Rather than forcing businesses to discard their current databases, SokoPulse acts as an intelligence layer. It connects to their existing data, extracts necessary telemetry, and applies machine learning to optimize pricing and prevent stock imbalances.

2. Problem Statement

Distributors and retailers lack a proactive, competitor-aware system to dynamically optimize commodity prices in real-time. Existing solutions fail to provide actionable market intelligence, forcing businesses into two risky extremes.

First, businesses risk overstocking heavy commodity assets. This ties up significant capital and incurs massive warehousing costs. Second, understocking leads to failed deliveries to retail networks, causing them to lose key accounts to competing distributors.

The core issue stems from the limitations of the software currently in use. Legacy ERPs act as internal-only ledgers without any concept of competitor pricing or demand elasticity. Basic POS systems rely on static, hardcoded rules that cannot adapt to seasonality or supplier reliability. Because of this, independent distributors are forced to make critical pricing decisions blindly, making it difficult to compete against tech-backed proprietary networks.

3. Objectives

Primary Objectives (Must-Have):

1. Develop a Database Tap Connector to securely extract transaction and stock supply data from existing retail databases without requiring manual data entry.
2. Build a concurrent web scraper that extracts real-time competitor pricing telemetry using User-Agent rotation to prevent blocking.
3. Train and deploy a Random Forest Regressor to forecast product demand up to 30 days in advance based on historical sales velocity.

Secondary Objectives (Nice-to-Have): 4. Implement a Machine Learning Classifier decision engine that dynamically recommends business actions, such as raising margins or applying discount markdowns. 5.

Create a seamless dashboard interface for distributors to approve or reject autonomous pricing recommendations.

4. Literature Review

The current landscape of retail software in the Kenyan market reveals a distinct divide between independent distributors and heavily funded tech-enabled networks.

Legacy Systems and Software Barriers: A diagnostic of the Kenyan entrepreneurial ecosystem by the World Bank (2023) highlights that while Micro, Small, and Medium Enterprises (MSMEs) dominate the economy, their integration into digital value chains is hindered by prohibitive software costs and a lack of scalable tools. Traditional ERPs and basic POS systems are reactive. They operate on static threshold tracking and fail to incorporate external market dynamics. Furthermore, a market assessment on supply chain finance (World Bank, 2022) notes that MSMEs face severe capital constraints; tying up capital in slow-moving inventory due to poor forecasting directly threatens their survival.

Proprietary Innovation in the Market: Conversely, platforms like Twiga Foods have demonstrated the immense value of data-driven B2B supply chains in Kenya. By directly integrating mobile telemetry and aggregating demand, they drastically reduce post-harvest losses and optimize pricing (GSMA, 2020). As noted by Ng'weno and Porteous (2018), integrating the informal sector with modern digital tools is the present and future of African commerce. However, these proprietary systems lock out independent distributors. SokoPulse addresses this by democratizing machine learning access, providing an open middleware alternative.

Machine Learning in Dynamic Pricing: In the domain of pricing optimization, Athey and Imbens (2019) establish that machine learning algorithms drastically improve predictive accuracy over traditional econometric models. Specifically, the Random Forest algorithm (Breiman, 2001) effectively handles the non-linear relationships found in retail pricing telemetry without overfitting. By leveraging modern frameworks like Scikit-learn (Pedregosa et al., 2011) and Pandas for data structuring (McKinney, 2010), these models can be deployed locally and cost-effectively, forming the computational core of SokoPulse.

5. Proposed System Architecture

SokoPulse uses a distributed, modular architecture to ensure non-intrusive integration and resilience.

Components and Data Flow:

1. **Database Tap (Data Ingestion):** Connects dynamically to the retailer's existing database (e.g., PostgreSQL, MySQL, SQLite), discovers schemas, and syncs transaction data.
2. **Web Scraper (Market Intelligence):** Runs concurrently to crawl competitor websites, bypassing bot blocks using a thread pool and User-Agent rotation. It extracts pricing metadata via JSON-LD.
3. **Background Processing (Task Queue):** Handled by Celery and Redis to ensure scraping and database syncing occur asynchronously without blocking the main application.
4. **Machine Learning Engine:** The Random Forest models consume the merged internal and external data to output forecasts and action classifications.
5. **Frontend/Backend:** A Django REST API serves the processed intelligence to a React/TypeScript frontend (built with Vite and TanStack Router) for the distributor to review.

Deployment Environment: The backend and ML models will be hosted on Railway, while the frontend will be deployed on Vercel. CI/CD pipelines via GitHub Actions will automate testing and deployment.

6. Methodology

The project will follow the **Agile/Scrum** development model, broken into two-week sprints.

Phases:

- **Requirements & Design:** Mapping out the Database Tap schema discovery algorithms and scraper targets.
- **Development:** Building the backend APIs, integrating the scraper, and training the Random Forest models.
- **Testing:**
 - *Unit tests:* PyTest for the Django backend and Python scraper; Jest for the React frontend components.
 - *Integration tests:* Validating data flow between the Database Tap, Redis task queue, and ML engine.
- **Deployment:** CI/CD rollout to Vercel and Railway.

Machine Learning Evaluation: The Random Forest Regressor will be evaluated using a chronological train-test split to prevent data leakage. Accuracy will be measured using Mean Absolute Error (MAE).

Success Criteria:

1. The Database Tap successfully synchronizes 10,000+ rows of legacy data in under 2 minutes without dropping records.
2. The web scraper successfully rotates User-Agents and retrieves competitor pricing data from target URLs with a 95% success rate.
3. The Random Forest model forecasts 30-day demand with a Mean Absolute Error (MAE) of 10 units or less on the test dataset.
4. The system seamlessly falls back to a 7-day rolling average algorithm when simulated network outages disrupt the scraper.

7. Timeline

The project will be executed over a 16-week timeline, tracked via GitHub Projects (Kanban).

Phase	Duration	Deliverables
Weeks 1-3	3 Weeks	Proposal submission, architecture design, and database modeling.
Weeks 4-6	3 Weeks	Development of the Database Tap Connector and concurrent web scraper.
Weeks 7-9	3 Weeks	Data collection, cleaning, and training of the Random Forest models.
Weeks 10-12	3 Weeks	Backend API development (Django) and task queue setup (Celery/Redis).
Weeks 13-14	2 Weeks	Frontend dashboard development (React/Vite) and integration.
Weeks 15-16	2 Weeks	End-to-end testing, deployment, and final report compilation.

8. Budget

The project prioritizes cost-efficiency by utilizing free-tier cloud services where possible.

- **Hosting (Frontend):** Vercel (Free Tier) - KES 0
- **Hosting (Backend & DB):** Railway Developer Plan - Approx. KES 1,500/month (KES 6,000 total)
- **Machine Learning Training:** Local compute (CPU) - KES 0
- **Proxy Network for Scraper (Optional):** KES 2,500/month (KES 10,000 total)
- **Domain Name:** Namecheap (.com or .co.ke) - Approx. KES 1,500/year

Estimated Total Cost: KES 17,500

9. Ethical Considerations

Data Privacy and Security: SokoPulse connects to sensitive business databases. The Database Tap will use read-only database user permissions to ensure legacy data cannot be accidentally overwritten. All transmitted data between the tap and our servers will be encrypted via TLS. We will not collect personally identifiable information (PII) of end consumers; we only sync anonymized commodity and transaction data.

Bias and Fairness: The Random Forest models will be continuously audited to ensure the pricing recommendations do not aggressively price-gouge during periods of natural supply chain crisis, prioritizing sustainable margins over exploitative spikes.

Open Source Compliance: The project utilizes open-source libraries (e.g., Django, Scikit-learn, React). All software will comply with their respective MIT and BSD licenses, and proper attribution will be maintained.

10. References

Athey, S., & Imbens, G. W. (2019). Machine learning methods that economists should know about. *Annual Review of Economics*, 11, 685-725.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

GSMA. (2020). *AgriTech in Kenya: The role of mobile in agriculture value chains*. GSMA AgriTech Programme.

McKinney, W. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, 51-56.

Ng'weno, A., & Porteous, D. (2018). Let's be real: The informal sector and the gig economy are the future, and the present, of work in Africa. *Center for Global Development*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

World Bank. (2022). *Kenya - Supply Chain Finance: Market Assessment*. Washington, DC: World Bank Group.

World Bank. (2023). *Entrepreneurship Ecosystems and MSMEs in Kenya: Strengthening Businesses in the Aftermath of the Pandemic*. Washington, DC: World Bank Group.

(Note: No AI tools were used to write the sentences or paragraphs in this proposal. AI was only utilized for structural brainstorming and grammar verification in accordance with CIT 4299 guidelines).